Sanskrit to English Neural Machine Translation

NLU Assignment 2

Beladiya Nikunj Bhupatbhai

Roll No: G25AIT1203

1. Introduction

The task is to translate Sanskrit sentences to English with a seq2seq model, on a corpus of 10k training pairs, 1k dev and 1k test. Sanskrit is a challenging source language: morphologically rich, only a small corpus available, and this dataset mixes scriptural verse with software tutorial lines ("I will click on Cancel").

The model I've been using is AI4Bharat's **IndicTrans2** (indic-en distilled, 200M), an encoder-decoder Transformer whose training data already includes Sanskrit (`san_Deva`) to English (`eng_Latn`). It gets a test BLEU of 0.146 zero-shot, with no training from my side at all, and 0.211 after fine-tuning on the 10k pairs.

The pre-trained models I use (disclosure): `ai4bharat/indictrans2-indic-en-dist-200M` (MIT license), which I fine-tune; and RoBERTa-large, which the `bert-score` package downloads to calculate the BERTScore metric. All processing happens on Colab's T4 GPU. No external APIs.

2. Architecture

IndicTrans2 is a standard 211,780,608 parameter Transformer encoder-decoder. Input is Sanskrit text, output is English. The Sanskrit gets encoded with self-attention over subword embeddings, and the decoder then runs masked self-attention plus cross-attention into the encoder output to generate the English autoregressively. The source vocabulary covers Indic scripts while the target vocabulary is English; the two sides are kept separate, which is unusual. The model uses the plain `eager` attention implementation since the T4 cannot support FlashAttention 2.

Decoding is beam search, 5 beams, `max_length=256`. Unlike greedy decoding, beam search retains the five best partial translations at each step, which matters most on the longer scriptural sentences where one early error can never be made up for.

3. Training procedure

Preprocessing. Each sentence gets NFC unicode normalization and whitespace cleanup. The CSVs have a BOM, so they are read with `utf-8-sig` and joined on `Source_id`. The data was clean in the sense that there were no nulls, duplicate rows or empty rows.

Tokenization. The `IndicTransToolkit` processor appends the language tag pair and applies the model's own preprocessing; the pre-trained SentencePiece tokenizer is then used to create subwords. The tokenizer was left as is. The whole idea of using a pre-trained model is to reuse its vocabulary.

Hyperparameters. Fine-tuning uses HuggingFace `Seq2SeqTrainer`:

Setting	Value
epochs	3
learning rate	3e-5, warmup ratio 0.1
batch size	8 (x2 gradient accumulation = effective 16)

Setting	Value
weight decay	0.01
precision	fp32 weights + fp16 mixed precision
checkpointing	per epoch, best dev-loss model reloaded at the end

One bug worth noting: the model loads in fp16 for fast inference (fp16 inference is fine, fp16 training is not), and AMP training fails on fp16 weights ("Attempting to unscale FP16 gradients"). The solution is to cast to fp32 prior to training and back to fp16 afterwards.

Regularization. Weight decay, a small learning rate, and only 3 epochs. With 10k examples against 200M parameters, restraint is the main regularizer. The training loss decreased steadily within the 3 epochs with dev loss following (Trainer log in the notebook); dev BLEU moved from 0.1645 to 0.2172.

4. Results

Single end-to-end run, Colab T4, beam 5, 1,000 sentences per split.

Model	Split	corpus BLEU	avg sentence BLEU	BERTScore F1	inference time
zero-shot	dev	0.1645	0.1523	0.5278	70.5 s
zero-shot	test	0.1458	0.1379	0.5113	62.7 s
fine-tuned	dev	0.2172	0.2094	0.5771	103.7 s
fine-tuned	**test**	**0.2114**	**0.2076**	**0.5671**	**102.6 s**

BLEU here is NLTK's default `corpus_bleu` with no custom weights; the per-sentence average additionally uses smoothing method 1. BERTScore is F1 with `rescale_with_baseline=True`. Efficiency: 211,780,608 parameters, 102.6 s for the 1,000 test sentences, about 103 ms each. Fine-tuning made inference slower even in fp16, since the fine-tuned model writes longer outputs in a style closer to the references, and hence needs more decoding steps. The fine-tuned model wins on dev, and its test predictions are the submitted `submission.csv`.

5. Translation examples

SA = source, REF = reference, HYP = model output.

1. **SA:** एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति। **REF:** Eclipse also helps the programmer to find out errors. **HYP:** Eclipse also helps the programmer in error detection. Correct sense, alternate phrasing. BLEU penalizes this, BERTScore does not.
2. **SA:** बालः युष्मासु विश्वासं करोति । **REF:** Boy has belief on you all. **HYP:** Boy has faith in you all. The output is better English than the reference. Cases like this cap the maximum achievable BLEU.
3. **SA:** तदा, तत्स्वयं ड्रैवर निमित्तम् अन्वेष्यति। अहं 'Cancel' इत्यस्योपरि नुदामि। **REF:** Then it will automatically begin searching for drivers. I will click on Cancel. **HYP:** It will then search for its own driver. I will click on Cancel. Second clause perfect; in the first, तत्स्वयम् gets read as "its own" instead of "automatically", plausible but incorrect.
4. **SA:** अपरं द्वितीयमुद्रायां तेन मोचितायां द्वितीयस्य प्राणिन आगत्य पश्येति वाक् मया श्रुता। **REF:** "And when he had opened the second seal, I heard the second beast say, Come and see." **HYP:** "And I heard a voice saying The second beast cometh and cometh which he hath redeemed out of the second sea." The weakest

case. Who opened gets confused with who speaks, मुद्रा (seal) is misread, and "sea" is hallucinated. The most obvious failure mode is long sentences of scripture.

5. **SA:** वयम्, ओब्जेक्ट्स् स्वकीयान् स्थितीन् फील्ड्स् मध्ये स्टोर् कुर्वन्तीति ज्ञातवन्तः । **REF:** We know that objects store their individual states in fields. **HYP:** We have learnt that objects store their positions in fields. स्थिति can mean state or position; in the programming sense it should be states.
6. **SA:** प्रकृति का अवलोकन करने से आप आश्चर्य चकित हो सकते है । **REF:** Observing the nature helps create a sense of wonder in you. **HYP:** Observing nature can take you by surprise. This line is actually Hindi, not Sanskrit (dataset noise), but the model still manages a reasonable translation.

Overall, technical sentences translate well, scriptural verse is where BLEU gets lost, and a reasonable share of the "errors" are acceptable paraphrases.

6. Experiments

Experiment	Test BLEU	Test F1	Outcome
zero-shot IndicTrans2	0.1458	0.5113	strong baseline, no training
+ fine-tune 3 epochs on 10k	0.2114	0.5671	kept (best dev BLEU)

During development I also implemented a from-scratch alternative: a 3-layer Transformer (d_model 256, ~12M parameters) plus a GRU with Bahdanau attention as baseline, using an 8k SentencePiece BPE vocabulary trained on the training set only. This exercise demonstrated the importance of subwords here: 45% of Sanskrit word types were OOV on the dev/test sets, vs. 0% unknown tokens with BPE and byte fallback. I chose to submit the fine-tuned pre-trained model since 10k pairs is too small to get a from-scratch model to a similar standard, and quality constitutes 70% of the grade. Model selection is done with dev only; test never influenced any choice.

7. Discussion

I went with the 200M distilled variant over the IndicTrans2 1B or NLLB-600M as it is the smallest model with native support for Sanskrit, which keeps the parameter penalty in the efficiency score small, and its MIT license is clean. Beam width 5 proved sufficient so I did not tune it any further.

Things that got in the way: the model is gated on HuggingFace, so you need to log in with a token; the fp16/fp32 training crash in section 3; and the mixed-domain data, where there isn't any single output style that works for both a UI instruction and a Bible verse. What is still not perfect: long scriptural Sanskrit remains weak (example 4); the corpus has some non-Sanskrit lines that fine-tuning inherits (example 6); and 3 epochs of full fine-tuning is near the safe maximum before the model starts memorizing 10k pairs.

8. References

1. Gala et al., *IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages*, TMLR 2023.
2. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.
3. Papineni et al., *BLEU: a Method for Automatic Evaluation of Machine Translation*, ACL 2002 (NLTK implementation).
4. Zhang et al., *BERTScore: Evaluating Text Generation with BERT*, ICLR 2020 (`bert-score` package).
5. Wolf et al., *Transformers: State-of-the-Art Natural Language Processing*, EMNLP 2020.

6. AI4Bharat, IndicTransToolkit, github.com/VarunGumma/IndicTransToolkit.